Provenance of the three phase-2 models

All facts below were read from the pinned source files (`third_party/manifest.json` lists the SHA256 of every file) and from the papers. Paper facts were extracted from the arXiv HTML full text; the key tables were cross-checked against the PDFs. Nothing here is recalled from memory.

Summary

	SAGAD	SpaceGNN	CRoC
Venue	WWW 2026	ICLR 2025	ECAI 2025
Paper	arXiv:2603.08137v1, DOI 10.1145/3774904.3792454	arXiv:2502.03201v1	arXiv:2508.12278v2 (Table 10 exists only in v2)
Authors	Y. Liu, Q. Xie, Y. Chen, X. Jin, T. Zheng, B. Chong, T. He	X. Dong, X. Zhang, L. Chen, M. Yuan, S. Wang	S. Xie, D. S. H. Tam, W. C. Lau
Repository	github.com/Cloudy1225/SAGAD	github.com/xydong127/SpaceGNN	github.com/XsLangley/CRoC_ECAI2025
Pinned commit	`faddcbca117b475c8468e87a65c8428b616b694b` (2025-07-20, only commit)	`921c03ff879b239dab9b319b296fef3bc3bda2d2` (2025-02-05, only commit)	`70cb7c6c654f2e8a517d226be5235e700953a37d` (2025-10-13, 4 commits)
Licence	none detected by GitHub	none detected	none detected
Stated environment	torch 2.4.0, dgl 2.4.0+cu124, numpy 2.1.3, scipy 1.14.1	Python 3.7, torch 1.12.1, dgl 0.9.0, PyG 2.3.1 (PyG unused by model code)	"PyTorch, DGL, NumPy, SciPy, scikit-learn" (unpinned)
Native data	GADBench DGL graphs (10 datasets incl. Elliptic)	GADBench DGL graphs (9 datasets)	DGL FraudYelp/FraudAmazon, BWGNN T-Fin/T-Soc, DGraph-Fin
Native label setting	semi-supervised: 100 labels (20 anomalies) on GADBench splits	50 train + 50 validation labels	1% training labels (0.01% for T-Soc)
Native selection	best validation AUROC	validation AUC + Macro-F1 after epoch 25	best validation AUC

Because no licence was detected, the bundle does **not** redistribute the authors' code. `scripts/fetch_official.py` downloads each repository at its pinned commit and verifies it file by file.

SAGAD

Architecture (`model.py`)

1. Precompute Chebyshev propagations `T_0(x) … T_K(x)` with the normalised operator −D^{-1/2} A D^{-1/2} (`precompute_Tx` , from PolyGCL).
2. Build two learnable ChebNetII filters, low-pass (`initial_val` 2.0, decreasing coefficients) and high-pass (`initial_val` 0.0, increasing coefficients), to get Z_l and Z_h.
3. Build the input as `concat[X, mean over MRQ-selected 1-hop neighbours of X]` (`main.py` , from UniGAD's Rayleigh-quotient sampler).
4. Fuse with a node- and dimension-wise gate C = σ(gate(input)), giving Z = C ⊙ Z_l + (1 − C) ⊙ Z_h (`NodeDimGatedFusion`).
5. Classify with an MLP.

Objective (`main.py::run`)

- Weighted cross-entropy with class weight w = #normal / #anomalous over the training nodes.
- Plus a weighted BCE between the mean gate value of each node and its target: p_a for anomalies, p_n for normal nodes; anomalies weighted by w.

- Optimiser: Adam(lr, weight_decay). Training is full-batch.

Hyperparameters

`semi_train.conf.yaml` is the only configuration released. The paper gives only a search grid:

- K ∈ {2..5}
- lr ∈ {0.1, 0.01, 0.001}
- wd ∈ {0, 0.01, 1e-4}
- hidden ∈ {32, 64, 128}
- dropout 0–0.5
- MLP depth 1–3
- p_a, p_n ∈ [0, 1]
- up to 1000 epochs, patience 50

Code issues verified in this audit

- `--semi type=bool` makes the flag always True, and `full_train.conf.yaml` is absent.
- The MRQ sampler is a Python loop over every node, with one `add_edges` call per node. That is O(N·E) and impractical on T-Social.
- **Verified:** `mrqsample` mutates the caller's feature tensor in place on a cache miss (`tests/test_sagad_mrq.py`). The first official run on a dataset therefore trains on min-max-scaled T_0(x) and X, while later, cached runs do not.
- The paper's class-weight β is described as anomalous/normal, while the code uses normal/anomalous.

Paper results for sanity ranges (AUPRC / AUROC, semi-supervised)

Amazon 84.7/94.5, YelpChi 27.8/68.8, T-Finance 82.8/94.4, Elliptic 57.1/90.9, T-Social 71.8/95.3.

SpaceGNN

Architecture (`model.py`)

- Three `CurvGNN` s: hyperbolic (curvature c<0), spherical (c>0) and Euclidean (c=0).
- Each has `layer_num` branches; branch i reads `feature_i` , the i-step mean-propagated input.
- Each branch applies a curvature-aware edge-weighted `CurvLayer` (expmap → proj → logmap), and the branches are concatenated.
- Curvatures are learnable and initialised as N(−0.1, stdneg) and N(0.1, stdpos). The std values are drawn by `name.py` after `random.seed(1)` .

Objective (`main.py`)

- NLL on (1−β)((1−α)·log p_hyp + α·log p_sph) + β·log p_euc.
- Adam(lr), no weight decay.
- Mini-batches of 50 training nodes with full one-hop blocks.
- At evaluation the exponentiated heads are mixed the same way (paper Algorithm 4).

Hyperparameters

Paper Table 4 equals `name.py PARAMETERS`; hidden size is 128 and batch size 50. Search grid:

- lr ∈ {1e-3, 1e-4}
- hidden ∈ {32, 64, 128, 256}
- layers 1–6
- dropout ∈ {0, 0.05, 0.1}
- α, β ∈ {0, 0.5, 1}

Code issues verified

- The validation call passes `(beta, alpha)` instead of `(alpha, beta)` (main.py line 149).
- The script never uses a GPU.
- Elliptic and FDCompCN (absent from `PARAMETERS`) crash on undefined `stdneg` / `stdpos`.
- The code reports no AUPRC.
- **Verified:** DGL's full-neighbour sampler returns call-dependent edge orders, and the hyperbolic head amplifies the resulting float32 noise to 4.7e-4 between two predictions of one model.

Paper results (AUC / F1, 50/50 labels)

Amazon 0.9331/0.8935, YelpChi 0.6566/0.5719, T-Finance 0.9400/0.8723, T-Social 0.9392/0.7571.

CRoC

Architecture (`models.py`)

- GraphSAGE- or GIN-style relation-aware convolutions (`RelSAGEConv`, `RelGINConv`) that concatenate a projected one-hot edge type with each neighbour's features.
- `_Large` variants add a 2-layer input MLP and a 2-layer predictor; `main.py` uses them for every dataset except yelp and amazon.

Objective (`trainers.py::train_epoch`)

- Balanced batches: all training positives plus an equal resampled set of negatives (`DSDataset`).
- Each batch also draws q = 10,240 random non-training nodes.
- Context refactoring: features are mixed as α·x + (1−α)·x_random.
- Loss = CE(raw) + γ·CE(mixed) + η·InfoNCE. InfoNCE pairs raw and mixed embeddings of the sampled nodes, with k negatives and temperature 2.
- Optimiser: Adam(lr 0.003).

Hyperparameters

Table 10 (v2) gives only α/γ/η/hidden/#epoch per dataset:

	α	γ	η	#epoch
Yelp	0	0.2	0.5	300
Amazon	0.5	0.2	0.5	200

	α	γ	η	#epoch
T-Fin	0.5	0.2	0.2	200
T-Soc	0	1	0.5	200

Hidden size is 64 for all four. lr, layers, fanouts (10, 5), k and τ come from the paper text; bs, q, dropout, bn and wd are code defaults. The backbone per dataset is not stated: Amazon uses SAGE (from the README); the others are inferred (see `configs/croc.yaml`).

Code issues verified

- No `dgl.seed`, so sampling is not reproducible.
- Elliptic and FDCompCN are unsupported (`NotImplementedError`).
- The README's "python main.py reproduces Yelp" uses γ=1, whereas Table 10 gives γ=0.2 for Yelp.

Paper results (AUC / AP, 1% labels)

Yelp 83.57/50.15, Amazon 89.20/52.98, T-Fin 92.44/76.48, T-Soc 95.58/64.24.

Neither Elliptic nor FDCompCN appears in any of the three papers' experiments, except Elliptic in SAGAD.